

MODULE 4 • KUBERNETES INTERNALS

Scheduling and Controllers

How Kubernetes chooses a node and keeps workloads moving toward the desired state.



What We Will Explain in This Module

Follow a Pod from scheduling through controller action and recovery after node failure.

LESSON 1

Scheduling cycle

Follow one Pod from the queue to a binding.

LESSON 2

Placement rules

Resources, labels, taints and scoring.

LESSON 3

Pending Pods

Read the event and choose the correct check.

LESSON 4

Controller internals

Informer, queue, reconcile, backoff and Lease.

LESSON 5

Node failure

Node health, taints, timers and eviction.

LESSON 6

Lab proof

KWOK, FailedScheduling and TaintManagerEviction.

How Kubernetes Schedules a Pod

From an empty `spec.nodeName` to one selected node.

1 · Find an unscheduled
Pod

2 · Filter and score
nodes

3 · Bind the selected
node

A new Pod has no node yet

Stored in the API → waiting for node assignment

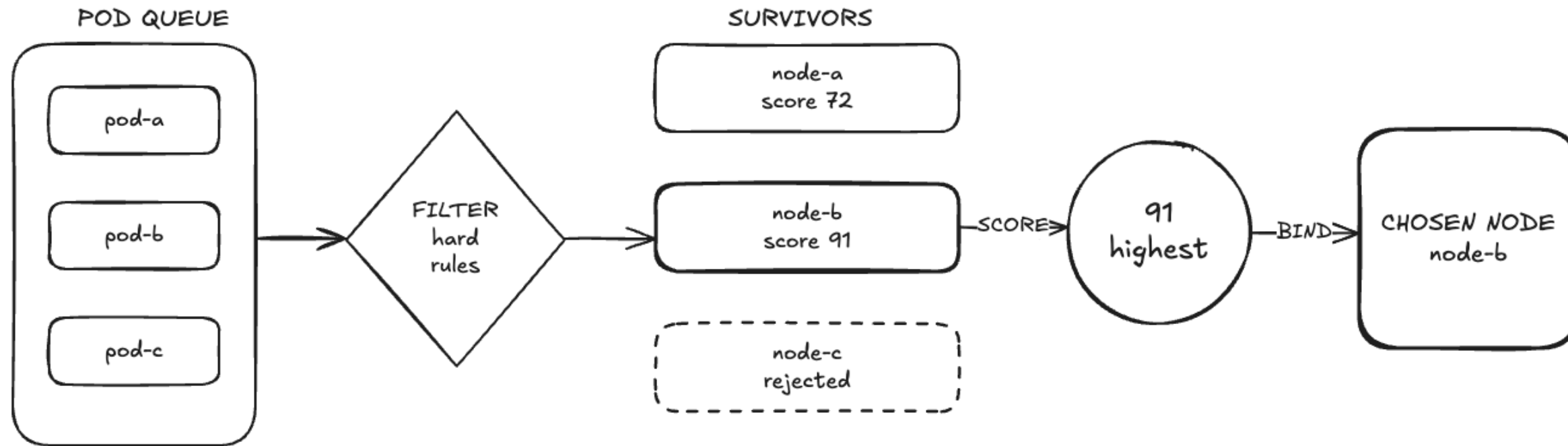
- 1 API server stores the Pod.
- 2 `spec.nodeName` is empty.
- 3 Scheduler watches for the Pending Pod.

```
apiVersion: v1
kind: Pod
metadata:
  name: api-worker
spec:
  # nodeName is not set
  containers:
  - name: worker
    image: registry.k8s.io/pause:3.9

status:
  phase: Pending
```

How a Scheduling Cycle Processes a Pod

Queue → filter → score → reserve and permit → bind



Filter: keep valid nodes. Score: rank survivors.

Filter

Remove nodes that fail a hard requirement.

Score

Rank only the nodes that survived filtering.

Reserve + Permit

Hold scheduler state and run any final approval.

Bind

Write the selected node to `spec.nodeName`.

Bind records the node name

API write → kubelet observes the assignment

BEFORE SCHEDULING

Pod is waiting

```
spec:
  nodeName: ""

status:
  phase: Pending
```

The scheduler can consider this Pod.

AFTER BIND

Pod has an assigned node

```
spec:
  nodeName: kwok-node-1

status:
  phase: Pending
```

The kubelet now owns the next step. The phase may remain Pending while containers start.

How Placement Rules Select a Node

Hard requirements remove nodes; scoring ranks what remains.

1 · Check resource fit

2 · Match labels and
taints

3 · Rank the survivors

Filters apply hard placement rules

Resource fit · label match · taint tolerance

Resources

CPU and memory requests fit one node.

Insufficient cpu

Labels and affinity

Required labels match the node.

didn't match node selector

Taints

Pod tolerates each NoSchedule taint.

untolerated taint

Resource requests must fit on one node

Requests—not live usage

```
resources:
  requests:
    cpu: "9000"
    memory: 4Gi
```

```
# Candidate node
Allocatable CPU: 32
Already requested: 28
Free for scheduling: 4
```

$28 \text{ used} + 9000 \text{ requested} > 32 \text{ allocatable}$

- The request must fit on one node.
- Adding many small nodes may not help one very large Pod.
- Fix the request or add a node that can satisfy it.

A node selector requires a matching label

Pod selector ↔ live node labels

POD REQUIREMENT

```
spec:
  nodeSelector:
    disktype: nvme
```

- This is a hard rule.
- A spelling error can block the Pod.

LIVE NODE LABELS

```
$ kubectl get nodes \
  -L disktype
```

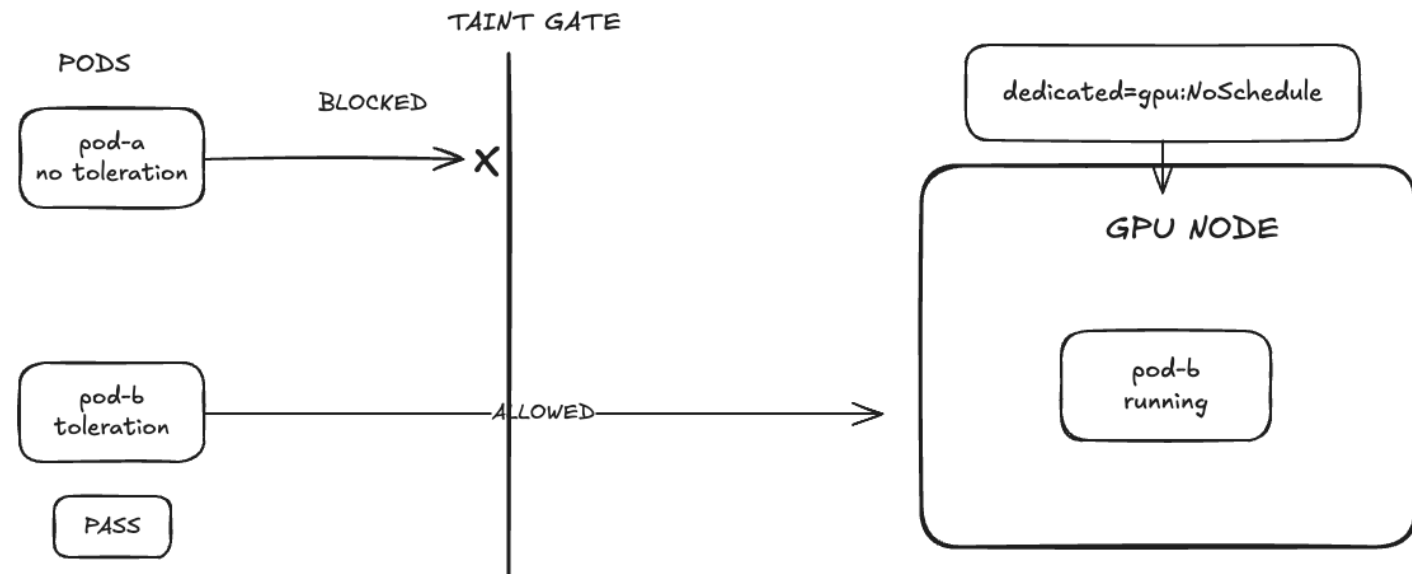
NAME	DISKTYPE
worker-0	ssd
kwok-node-1	<none>

Result

No node has disktype=nvme, so the Pod remains Pending.

How Taints and Tolerations Control Placement

Node taint ↔ Pod toleration



A toleration allows entry. It does not choose the node.

- 1 A taint is attached to a **node**.
- 2 A toleration is attached to a **Pod**.
- 3 A toleration permits placement; it does not choose the node.
- 4 NoExecute can also evict Pods already on the node.

```
dedicated=gpu:NoSchedule
```

```
tolerations:
```

- key: dedicated
- value: gpu
- effect: NoSchedule

Scoring ranks the nodes that remain

Only surviving nodes are scored

- Balance CPU and memory.
- Spread across topology domains.
- Select the highest total score.

1	worker-zone-b Good spread and enough free capacity	87
2	worker-zone-a Valid, but less balanced	72
3	worker-zone-c Valid, but lower topology score	60

How to Diagnose a Pending Pod

Use `FailedScheduling` to choose the next check.

1. Read the event

2. Inspect the rejected
rule

3. Fix only the cause

Start with the FailedScheduling event

Read event → inspect rule → change one thing

```
$ kubectl describe pod too-big -n m4-lab
```

Events:

Type	Reason	From
Warning	FailedScheduling	default-scheduler

0/5 nodes are available:

4 Insufficient cpu, 1 node had intolerated taint.

preemption: 5 Preemption is not helpful for scheduling.

1. Find the reason

Read each rejected-node count.

2. Check that rule

Inspect requests, labels or taints.

3. Make one change

Fix the exact rule that failed.

4. Verify again

Watch the event and Pod phase.

The resource request is too large

4 Insufficient cpu

```
$ kubectl get pod too-big -n m4-lab \
  -o jsonpath='{.spec.containers[*].resources.requests}'
```

```
{"cpu":"9000"}
```

```
$ kubectl describe node kwok-node-0 | grep -A5 Allocatable
Allocatable:
  cpu: 32
```

- Compare the request with node allocatable CPU.
- Preemption cannot create a larger node.
- Right-size the request or use a larger node.

The node selector matches nothing

```
4 node(s) didn't match Pod's node affinity/selector
```

```
$ kubectl get pod wrong-selector -n m4-lab \
  -o jsonpath='{.spec.nodeSelector}'
{"disktype":"nvme-that-does-not-exist"}
```

```
$ kubectl get nodes -L disktype
```

NAME	DISKTYPE
worker-0	<none>
kwok-node-1	<none>

- Compare the Pod selector with live node labels.
- Check spelling, case and node pool.
- Fix the selector or label the intended node.

The Pod cannot tolerate a taint

```
4 node(s) had untolerated taint
```

```
$ kubectl get node kwok-node-0 \
  -o jsonpath='{.spec.taints}'
```

```
[{"key":"kwok.x-k8s.io/node",
  "value":"fake",
  "effect":"NoSchedule"}]
```

```
$ kubectl get pod no-toleration -n m4-lab \
  -o jsonpath='{.spec.tolerations}'
[]
```

- Read the taint key, value and effect.
- Compare them with the Pod tolerations.
- Add a narrow toleration only when policy allows it.

Use the event to choose the next check

Reason → next check → focused repair

Insufficient cpu

Resources

request ↔ allocatable

**didn't match
selector**

Labels

selector ↔ live labels

untolerated taint

Policy

taint ↔ toleration

**preemption not
helpful**

Hard constraint

removing Pods cannot fix it

How Controllers Process Changes

Watch → cache → queue → reconcile → retry.

1 · Watch API changes

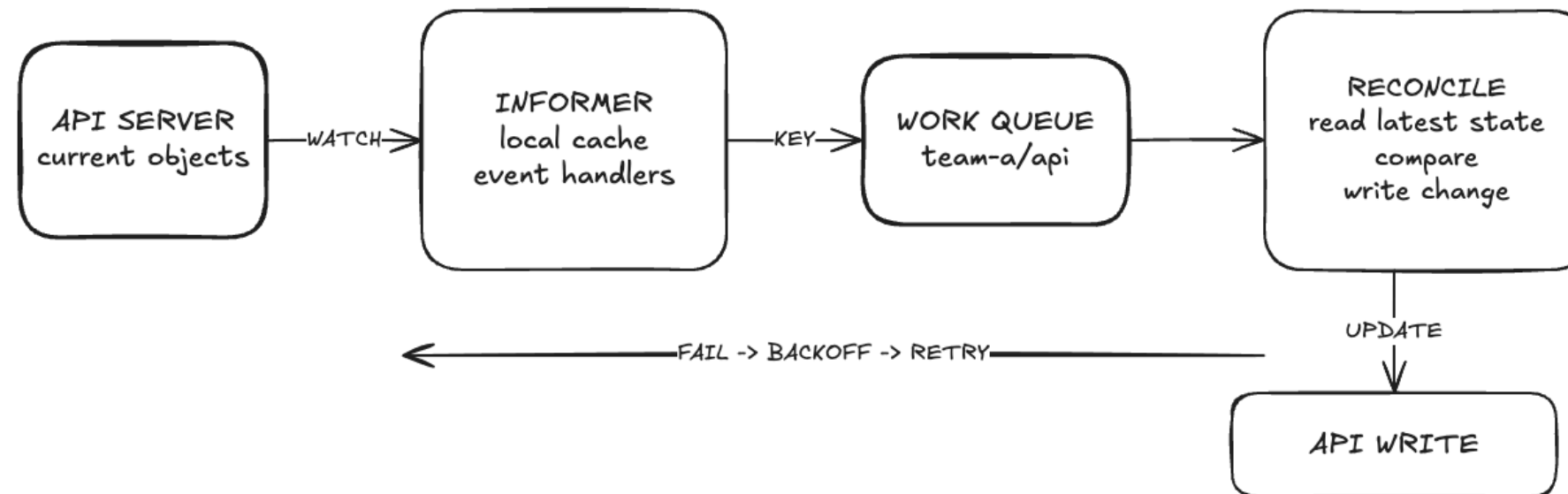
2 · Queue an object key

3 · Reconcile and retry

How a Controller Processes One Change

Watch → cache → queue → reconcile → write or retry

CONTROLLER LOOP



Events enqueue a key. Reconcile reads the latest object.

Informer

Watch API; update cache.

Work queue

Queue object key; deduplicate.

Reconcile

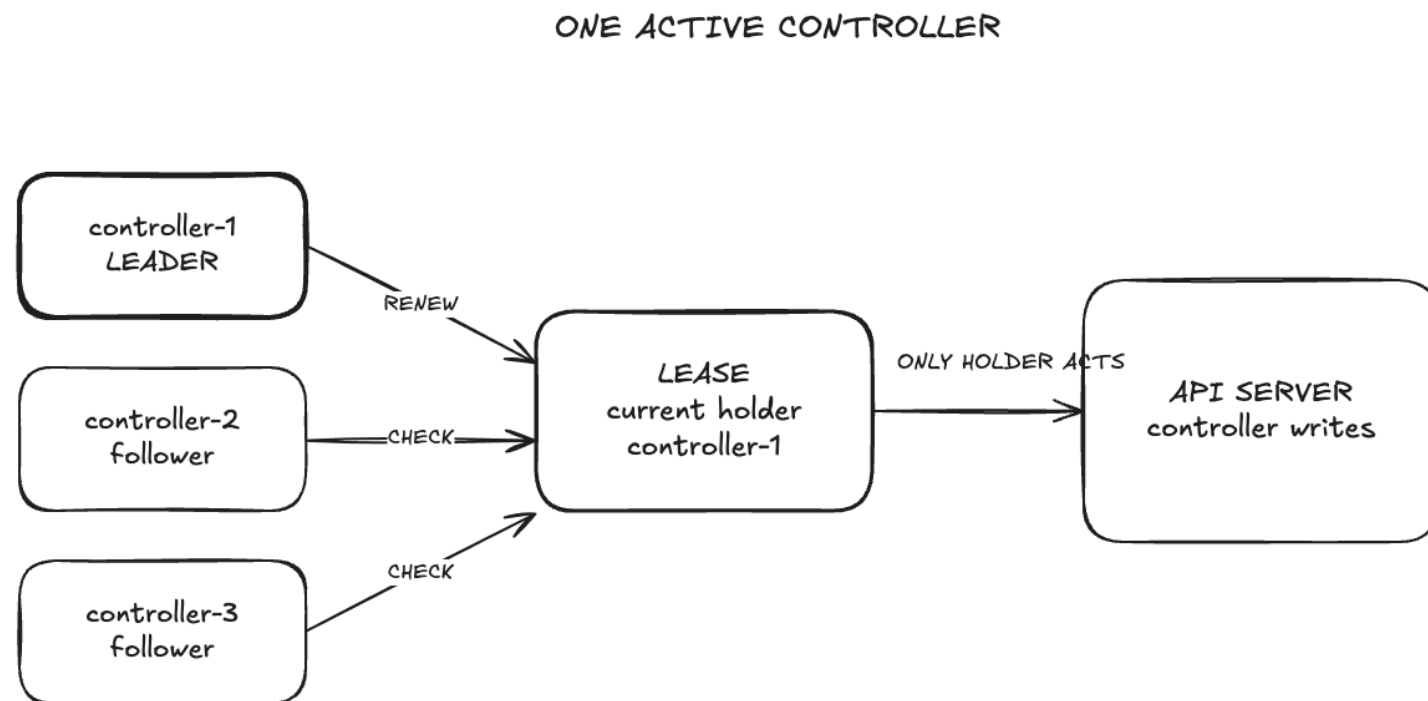
Read latest state; compare.

Backoff

Retry failed keys later.

How Leader Election Keeps One Replica Active

Lease holder writes · followers wait



Followers stay ready. A new holder takes over when the Lease expires.

- 1 Every replica checks the same leader-election Lease.
- 2 The current holder renews the Lease and processes work.
- 3 A follower takes over when the Lease expires.

One active writer · followers ready

What Happens When a Node Stops Reporting

Detection → system taints → eviction or recovery.

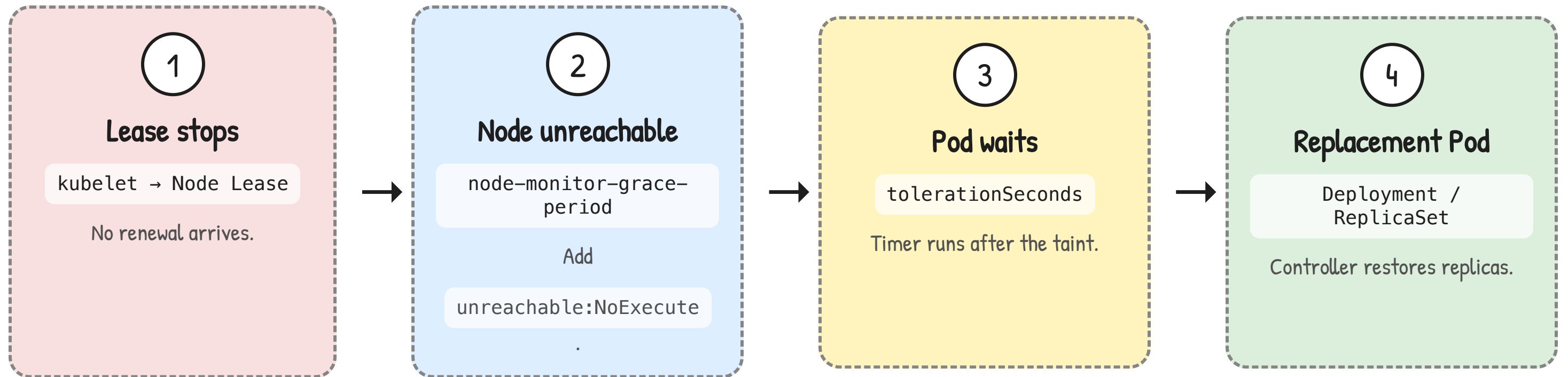
1 · Detect the lost Lease

2 · Add system taints

3 · Evict or recover

How Kubernetes Handles an Unreachable Node

Detection → NoExecute eviction → replacement on a healthy node



Lease stops → unreachable:NoExecute → tolerationSeconds expires → replacement

NoSchedule and NoExecute do different jobs

Placement effect vs eviction effect

NoSchedule

New Pods

Block unless tolerated

Existing Pods stay

PreferNoSchedule

New Pods

Avoid when possible

Existing Pods stay

NoExecute

All Pods

Block new · evict existing

Timer can delay eviction

Lab: `maintenance=cordon:NoExecute` → observe `TaintManagerEviction`

tolerationSeconds controls the eviction delay

Node recovers first ↔ timer expires first

NODE RETURNS FIRST

Pod remains bound

- Lease renews.
- System taint is removed.

TIMER EXPIRES FIRST

Pod is deleted

- Taint Manager evicts.
- Controller restores replicas.

Short timer: faster recovery · Long timer: fewer restarts during brief network loss

Prove the Model in the Lab

Create failures, inspect evidence, repair the policy.

1 · Create real failures

2 · Read events and X-Ray

3 · Repair and verify

Create and repair three Pending Pods

Create failure → read reason → repair one rule

too-big

Insufficient cpu

9000 → 100m

wrong-selector

selector mismatch

add the intended label

no-toleration

untolerated taint

add matching toleration

```
$ kubectl get pod too-big wrong-selector no-toleration -n m4-lab -o wide
```

NAME	READY	STATUS	NODE
too-big	1/1	Running	kwok-node-0
wrong-selector	1/1	Running	kwok-node-1
no-toleration	1/1	Running	kwok-node-2

Use KWOK to observe eviction

Fake Nodes · real scheduling state

- Create three fake nodes with `kwok-setup.sh`.
- Schedule `on-fake` with the required selector and toleration.
- Add `maintenance=cordon:NoExecute` to its node.
- Read the `TaintManagerEviction` event.

```
$ node=$(kubectl get pod on-fake -n m4-lab \
-o jsonpath='{.spec.nodeName}')

$ kubectl taint node "$node" \
maintenance=cordon:NoExecute

node/kwok-node-0 modified

$ kubectl get events -n m4-lab \
--field-selector reason=TaintManagerEviction

TaintManagerEviction  on-fake
Marking for deletion Pod m4-lab/on-fake
```

Review policy before changing it

A Pending Pod may be enforcing the correct platform rule.

- 1 **Requests:** measured usage + justified buffer
- 2 **Placement:** hard rules only for real requirements
- 3 **Isolation:** narrow taints and tolerations
- 4 **Priority:** understand the failed constraint first
- 5 **Recovery:** timer aligned with replacement capacity

Scheduling and Controllers

- 1 **Filter** removes invalid nodes; **score** ranks only the survivors.
- 2 **Bind** records the selected node in `spec.nodeName`.
- 3 **FailedScheduling** identifies the rule that rejected the Pod.
- 4 A toleration permits placement; it does not attract a Pod to the node.
- 5 **NoExecute** eviction uses a node-detection clock and a Pod toleration clock.

Next: observe these decisions in the Module 4 lab.